

# Contrato API — Adopción de Mascotas (v1.0 · cerrado)

**Estado: cerrado** el 29 de septiembre de 2026 (SCRUM-41), a partir de la propuesta v0.1 de Dev C, con Tomás, Santiago y Gustavo (ver 7. *Acta de cierre*). Es la fuente de verdad para el backend, el frontend y el adaptador de MongoDB: **cualquier cambio posterior sube la versión y se registra en 6. Historial de cambios**.

El backend cumple este contrato y lo verifica con pruebas de integración ( backend/tests/integration/ , incluida test\_contrato.py , que compara la tabla de endpoints con las rutas reales); el frontend ( frontend/src/api/httpAdapter.js ) y su mock usan exactamente estas formas.

- **Base URL (vista desde el navegador):** /api/v1
- ⚠ El nginx.conf del repo **quita el prefijo /api** antes de pasar al backend. Por eso **los routers de FastAPI deben montarse en /v1** (ej. app.include\_router(animals\_router, prefix="/v1") ). El proxy de Vite en desarrollo hace lo mismo.
- **Formato:** JSON:API 1.1
- **Content-Type (request y response):** application/vnd.api+json
- **IDs:** strings (ObjectId de MongoDB serializado). El dominio no conoce ObjectId : el adaptador de Mongo traduce.
- **Fechas:** ISO-8601 en UTC con sufijo Z (ej. 2026-09-28T14:02:11Z ).
- **Sin autenticación** en el alcance del taller. El "rol" (refugio / adoptante) se elige en el frontend.
- **Documentación interactiva:** http://localhost:8000/docs (Swagger, generada por FastAPI).

## 0. Modelo de dominio y reglas de negocio

### Entidades finales

| Entidad     | Atributos del dominio                                                                                                                                                                   | Estados                                 | Código      |
|-------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------|-------------|
| Refugio     | nombre, email, teléfono, dirección (ciudad, barrio, línea), fecha de creación                                                                                                           | —                                       | Refugio     |
| Animal      | refugio, nombre, especie, edad en meses, sexo, temperamento, salud (vacunado, esterilizado, desparasitado, notas), fotos (url, descripción), detalles por especie, fecha de publicación | disponible →<br>postulado →<br>adoptado | Animal      |
| Adoptante   | nombre, email (lo identifica, sin login), teléfono, ciudad, fecha de registro                                                                                                           | —                                       | Adoptante   |
| Postulación | animal, adoptante, mensaje, motivo de cierre, fecha de postulación, fecha de resolución                                                                                                 | pendiente →<br>aprobada o<br>rechazada  | Postulacion |

Implementación: `backend/app/domain/entities.py` (Pydantic). Puertos: `AnimalRepository` , `PostulacionRepository` , `RefugioRepository` , `AdoptanteRepository` ( `backend/app/application/ports/` ).

## Reglas de negocio

| #  | Regla                                                                                                                                                                                          | Dónde se aplica                                                                                                    | Si se incumple                              |
|----|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|---------------------------------------------|
| R1 | Solo se puede postular a un animal <b>no adoptado</b> ( <code>disponible</code> o <code>postulado</code> ).                                                                                    | <code>Animal.validar_postulable()</code>                                                                           | 409<br><code>ANIMAL_NO_DISPONIBLE</code>    |
| R2 | Un adoptante no puede tener <b>más de una postulación pendiente</b> sobre el mismo animal. Tras un rechazo puede volver a postular.                                                            | Caso de uso <code>Postular</code> + <code>existe_pendiente()</code> ; respaldo en el índice único parcial de Mongo | 409<br><code>POSTULACION_DUPLICADA</code>   |
| R3 | Al <b>aprobar</b> una postulación, el animal pasa a <code>adoptado</code> y <b>todas las demás pendientes</b> sobre ese animal se rechazan con <code>motivoCierre = cierre_automatico</code> . | Caso de uso <code>AprobarPostulacion</code>                                                                        | —                                           |
| R4 | La primera postulación pasa el animal de <code>disponible</code> a <code>postulado</code> .                                                                                                    | <code>Animal.registrar_postulacion()</code>                                                                        | —                                           |
| R5 | Si el refugio <b>rechaza la última pendiente</b> , el animal vuelve a <code>disponible</code> .                                                                                                | Caso de uso <code>RechazarPostulacion</code>                                                                       | —                                           |
| R6 | Solo una postulación <code>pendiente</code> se puede aprobar o rechazar.                                                                                                                       | <code>Postulacion</code> (método <code>_resolver</code> )                                                          | 409<br><code>POSTULACION_YA_RESUELTA</code> |
| R7 | Solo un animal <code>postulado</code> puede pasar a <code>adoptado</code> , y uno <code>adoptado</code> no vuelve a <code>disponible</code> .                                                  | <code>Animal.marcar_adoptado()</code> / <code>liberar()</code>                                                     | 409 <code>TRANSICION_INVALIDA</code>        |
| R8 | El estado del animal lo decide el dominio: al publicar siempre nace <code>disponible</code> (se ignora el que envíe el cliente).                                                               | Caso de uso <code>PublicarAnimal</code>                                                                            | —                                           |
| R9 | Registrar un adoptante con un email ya existente (sin distinguir mayúsculas) <b>reutiliza</b> ese adoptante.                                                                                   | Caso de uso <code>RegistrarAdoptante</code>                                                                        | 200 en vez de 201                           |

## 1. Recursos

`refugios`

| Atributo | Tipo   | Notas                    |
|----------|--------|--------------------------|
| nombre   | string | requerido                |
| ciudad   | string | requerido                |
| telefono | string | opcional                 |
| email    | string | requerido, formato email |

Solo lectura en la API (los refugios vienen de la semilla). Sin relaciones expuestas: los animales de un refugio se consultan con `GET /animals?filter[refugio]={id}`.

#### animals

| Atributo        | Tipo              | Notas                                                                                                                                      |
|-----------------|-------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| nombre          | string            | requerido, 1–60 caracteres                                                                                                                 |
| especie         | enum              | perro · gato · ave · otro                                                                                                                  |
| edadMeses       | int               | $\geq 0$                                                                                                                                   |
| sexo            | enum              | macho · hembra                                                                                                                             |
| temperamento    | string[]          | ej. ["juguetón", "sociable"]; máx. 15 rasgos de $\leq 40$ caracteres                                                                       |
| salud           | object            | { vacunado: bool, esterilizado: bool, desparasitado: bool, notas: string } (todos opcionales)                                              |
| fotos           | string[]          | URLs <code>http(s)://</code> o <code>data:image/...</code> ; máx. 10. En la entrada también se acepta { url, descripcion }                 |
| detallesEspecie | object            | campos libres según especie (ej. perro: tamaño, ave: puedeHablar), máx. 20 claves — <b>aquí se aprovecha el modelo documental de Mongo</b> |
| estado          | enum              | disponible · postulado · adoptado — <b>solo lectura</b> , lo cambia el dominio                                                             |
| publicadoEn     | datetime ISO-8601 | solo lectura                                                                                                                               |

Relaciones: refugio (to-one, requerida al crear). Las postulaciones de un animal no se exponen como relación (YAGNI): el panel las consulta por refugio.

#### adoptantes

| Atributo | Tipo   | Notas                                                                                  |
|----------|--------|----------------------------------------------------------------------------------------|
| nombre   | string | requerido, 2–80 caracteres                                                             |
| email    | string | requerido, único (sin distinguir mayúsculas), formato <code>usuario@dominio.tld</code> |
| telefono | string | requerido, 7–13 dígitos sin espacios                                                   |
| ciudad   | string | requerido, $\geq 2$ caracteres                                                         |

#### postulaciones

| Atributo     | Tipo            | Notas                                                                                  |
|--------------|-----------------|----------------------------------------------------------------------------------------|
| mensaje      | string          | requerido, 10–500 caracteres (por qué quiere adoptar)                                  |
| estado       | enum            | pendiente · aprobada · rechazada                                                       |
| motivoCierre | string   null   | "rechazada_por_refugio" o "cierre_automatico" — permite mostrar en UI por qué se cerró |
| creadaEn     | datetime        | solo lectura                                                                           |
| resueltaEn   | datetime   null | solo lectura                                                                           |

Relaciones: animal (to-one), adoptante (to-one).

## Nota sobre nombres (Mongo ↔ API)

**Decisión cerrada:** el adaptador de salida de Mongo traduce; ni el dominio ni la API dependen de cómo guarda Mongo. La semilla de Tomás guarda en Mongo con **snake\_case** (edad\_meses, detalles\_especie, fecha\_publicacion, colección animales) y fotos como { url, descripcion }. Lo ideal es que el **adaptador de salida** traduzca a los nombres del contrato, así el dominio y la API no dependen de cómo guarda Mongo. De todas formas **el frontend acepta ambas formas** (snake\_case o camelCase, type: "animals" o "animales", fotos como URL u objeto, motivo\_rechazo o motivoCierre), así que no bloquea a nadie. Traducciones que debe hacer el adaptador de Mongo: ObjectId ↔ string, colección animales ↔ tipo animals, y el texto libre motivo\_rechazo de la semilla ↔ el enum motivoCierre.

## 2. Endpoints

| Método | Ruta                                                         | Uso                                                           | Pantalla               |
|--------|--------------------------------------------------------------|---------------------------------------------------------------|------------------------|
| GET    | /refugios                                                    | Listar refugios                                               | Selector de rol        |
| GET    | /refugios/{id}                                               | Detalle                                                       | —                      |
| GET    | /animals                                                     | Listado con filtros y paginación                              | Catálogo               |
| GET    | /animals/{id}?include=refugio                                | Ficha                                                         | Ficha del animal       |
| POST   | /animals                                                     | Refugio publica animal                                        | Panel refugio          |
| POST   | /adoptantes                                                  | Registro simple de adoptante (201; 200 si el email ya existe) | Formulario postulación |
| GET    | /adoptantes/{id}                                             | Detalle                                                       | —                      |
| POST   | /postulaciones                                               | Adoptante postula                                             | Formulario postulación |
| GET    | /postulaciones?filter[refugio]={id}&include=animal,adoptante | Postulaciones del refugio                                     | Panel refugio          |
| GET    | /postulaciones?filter[adoptante]={id}&include=animal         | "Mis postulaciones"                                           | Seguimiento            |

| Método | Ruta                | Uso                           | Pantalla      |
|--------|---------------------|-------------------------------|---------------|
| PATCH  | /postulaciones/{id} | Aprobar / rechazar ( estado ) | Panel refugio |

Todas las rutas van bajo `/v1` en el backend ( `/api/v1` desde el navegador). `GET /health` (fuera de `/v1`) es de infraestructura y no usa JSON:API.

## Reglas comunes de las respuestas

- `POST` exitoso: **201** con el recurso creado y cabecera `Location` (animals, adoptantes). `POST /animals` incluye el refugio en `included`; `POST /postulaciones` incluye el animal y el adoptante.
- Listas: `meta.total` siempre; `GET /animals` además trae `links` de paginación.
- `include` solo acepta las relaciones indicadas en cada endpoint; otra  $\rightarrow$  400.

## Parámetros de `GET /postulaciones`

- Obligatorio **uno** de: `filter[refugio]={id}` o `filter[adoptante]={id}` (sin ninguno  $\rightarrow$  400). Si vienen ambos, se intersectan.
- Opcional: `filter[estado]=pendiente|aprobada|rechazada`.
- `include=animal,adoptante` (cualquiera de los dos).

## Parámetros de `GET /animals`

- `filter[estado]=disponible`
- `filter[especie]=perro`
- `filter[refugio]={id}`
- `page[number]=1&page[size]=12` (tamaño por defecto 12, máximo 50; fuera de rango  $\rightarrow$  400)
- `include=refugio`

La respuesta trae `meta.total` y `links.first/prev/next/last`.

## 3. Ejemplos

### Postular — `POST /postulaciones`

```
{
  "data": {
    "type": "postulaciones",
    "attributes": { "mensaje": "Tengo patio grande y experiencia con perros." },
    "relationships": {
      "animal": { "data": { "type": "animals", "id": "66f1a0c2e4b0a1" } },
      "adoptante": { "data": { "type": "adoptantes", "id": "66f1a0c2e4b0z9" } }
    }
  }
}
```

**201 Created**

```
{
  "data": {
    "type": "postulaciones",
    "id": "66f1b3d9aa01",
    "attributes": { "mensaje": "Tengo patio...", "estado": "pendiente", "motivoCierre": null,
                    "creadaEn": "2026-09-28T14:02:11Z", "resueltaEn": null },
    "relationships": {
      "animal": { "data": { "type": "animals", "id": "66f1a0c2e4b0a1" } },
      "adoptante": { "data": { "type": "adoptantes", "id": "66f1a0c2e4b0z9" } }
    }
  }
}
```

### Aprobar — PATCH /postulaciones/{id}

```
{ "data": { "type": "postulaciones", "id": "66f1b3d9aa01", "attributes": { "estado": "aprobada"
} } }
```

**200 OK** — devuelve la postulación aprobada e incluye en `included` el animal (ya `adoptado`) y las demás postulaciones que se cerraron automáticamente. Así el panel del refugio se actualiza con una sola respuesta:

```
{
  "data": { "type": "postulaciones", "id": "66f1b3d9aa01", "attributes": { "estado":
"aprobada", "...": "..." } },
  "included": [
    { "type": "animals", "id": "66f1a0c2e4b0a1", "attributes": { "estado": "adoptado", "...":
"..." } },
    { "type": "postulaciones", "id": "66f1b3d9aa02", "attributes": { "estado": "rechazada",
"motivoCierre": "cierre_automatico" } }
  ],
  "meta": { "postulacionesCerradasAutomaticamente": 1 }
}
```

## 4. Errores (formato JSON:API)

```
{
  "errors": [{
    "status": "409",
    "code": "ANIMAL_NO_DISPONIBLE",
    "title": "El animal no está disponible",
    "detail": "Luna ya fue adoptada.",
    "source": { "pointer": "/data/relationships/animal" }
  }]
}
```

| HTTP | code                    | Cuándo                                                                                                          |
|------|-------------------------|-----------------------------------------------------------------------------------------------------------------|
| 400  | SOLICITUD_MALFORMADA    | JSON inválido o <code>type</code> incorrecto                                                                    |
| 404  | RECURSO_NO_ENCONTRADO   | id inexistente                                                                                                  |
| 409  | ANIMAL_NO_DISPONIBLE    | postular a animal <code>adoptado</code>                                                                         |
| 409  | POSTULACION_DUPLICADA   | el adoptante ya tiene una <code>pendiente</code> sobre ese animal                                               |
| 409  | POSTULACION_YA_RESUELTA | aprobar/rechazar una postulación que no está <code>pendiente</code>                                             |
| 409  | TRANSICION_INVALIDA     | cambio de estado del animal que su ciclo de vida no permite                                                     |
| 409  | ID_NO_COINCIDE          | en un <code>PATCH</code> , <code>data.id</code> distinto del id de la URL                                       |
| 405  | METODO_NO_PERMITIDO     | verbo HTTP no soportado en esa ruta                                                                             |
| 415  | CONTENT_TYPE_INVALIDO   | falta <code>application/vnd.api+json</code>                                                                     |
| 422  | VALIDACION              | campo inválido; un error por campo con <code>source.pointer</code> (ej. <code>/data/attributes/mensaje</code> ) |
| 500  | ERROR_INTERNO           | no esperado (no expone detalles internos)                                                                       |

Los errores de parámetros ( `filter[...]` , `page[...]` , `include` ) son 400 con `source.parameter` en vez de `source.pointer` .

**Implementación** ( `backend/app/adapters/inbound/api/errors.py` ): las excepciones de dominio ( `app/domain/exceptions.py` ) se traducen a estos códigos con un `exception_handler` de FastAPI; el dominio no conoce HTTP, solo el adaptador de entrada. Una subclase nueva hereda el código de su padre, y una regla sin mapeo responde 409 `REGLA_DE_NEGOCIO` (nunca 500).

## Las dos validaciones de la postulación

| Regla                                 | Dónde se valida                                                                                                                                                                     | Respuesta                                                                                                       |
|---------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| No postular a un animal no disponible | <code>Animal.validar_postulable()</code> (dominio). Solo <code>adoptado</code> bloquea; <code>postulado</code> sigue aceptando (ver 5.1).                                           | 409 <code>ANIMAL_NO_DISPONIBLE</code> , <code>detail</code> : "Luna ya fue adoptada."                           |
| No duplicar una postulación activa    | Caso de uso <code>Postular</code> , con <code>PostulacionRepository.existe_pendiente()</code> . Solo cuenta la <code>pendiente</code> : tras un rechazo se puede volver a postular. | 409 <code>POSTULACION_DUPLICADA</code> , <code>detail</code> : "Ya tienes una postulación pendiente para Luna." |

La regla 1 se evalúa antes que la 2, y un rechazo no guarda nada ni cambia el estado del animal. En MongoDB, la regla 2 tiene además un respaldo en el índice único parcial `postulacion_pendiente_unica`, para el caso de dos peticiones simultáneas.

## 5. Decisiones cerradas

Las cuatro decisiones abiertas de la propuesta v0.1 quedaron así, y están implementadas y probadas:

| #   | Decisión                                                                                 | Resolución                                                                                                                                                                                                                       | Evidencia                                                                  |
|-----|------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------|
| 5.1 | Estado <code>postulado</code> vs. "solo se postula a un animal <code>disponible</code> " | Se puede postular en <code>disponible</code> o <code>postulado</code> ; solo <code>adoptado</code> bloquea. <code>postulado</code> = "tiene al menos una pendiente". Sin esto nunca habría dos pendientes y R3 no se ejecutaría. | R1, R4 ·<br><code>test_animal_postulado_si_acepta_mas_postulaciones</code> |
| 5.2 | ¿Qué pasa si el refugio rechaza todas las pendientes?                                    | El animal vuelve a <code>disponible</code> .                                                                                                                                                                                     | R5 ·<br><code>test_rechazar_la_unica_pendiente_libera_al_animal</code>     |
| 5.3 | Registro de adoptante                                                                    | Registro mínimo en el formulario de postulación, sin login. Si el email ya existe se reutiliza el adoptante (200).                                                                                                               | R9 ·<br><code>test_registrar_adoptante_y_reutilizar_por_email</code>       |
| 5.4 | Fotos                                                                                    | URLs externas (sin subida de archivos). Solo <code>http(s)://</code> o <code>data:image/...</code> , para no pintar <code>javascript:</code> en <code>&lt;img src&gt;</code> .                                                   | <code>test_foto_con_url_peligrosa_o_invalida_es_422</code>                 |

Pendiente para el adaptador de MongoDB (no cambia el contrato): traducir `DuplicateKeyError` del índice `postulacion_pendiente_unica` a `PostulacionDuplicada`, para que dos postulaciones simultáneas respondan 409 y no 500.

## 6. Historial de cambios

| Versión | Fecha      | Cambios                                                                             |
|---------|------------|-------------------------------------------------------------------------------------|
| v0.1    | sept. 2026 | Propuesta de Dev C: recursos, endpoints, ejemplos, errores y 4 decisiones abiertas. |

| Versión | Fecha         | Cambios                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|---------|---------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| v1.0    | 29 sept. 2026 | <b>Cierre.</b> Modelo de dominio y reglas R1–R9; decisiones 5.1–5.4 resueltas; <code>salud.desparasitado</code> ; validación y topes de <code>fotos</code> , <code>temperamento</code> y <code>detallesEspecie</code> ; validaciones del adoptante; <code>filter[estado]</code> y filtro obligatorio en <code>GET /postulaciones</code> ; 200 al reutilizar adoptante; <code>included</code> y <code>Location</code> en los <code>POST</code> ; se retiran las relaciones to-many no usadas ( <code>refugio.animals</code> , <code>animal.postulaciones</code> ); códigos <code>TRANSICION_INVALIDA</code> , <code>ID_NO_COINCIDE</code> , <code>METODO_NO_PERMITIDO</code> . |

## 7. Acta de cierre

Reunión de cierre del contrato antes de repartir el trabajo (SCRUM-41). Cada integrante confirma que su parte se construye contra esta versión.

| Integrante         | Rol                  | Parte que depende del contrato                                         | Conforme                 |
|--------------------|----------------------|------------------------------------------------------------------------|--------------------------|
| Gustavo Aguilar    | Dev A · backend      | Dominio, puertos, casos de uso, endpoints y errores                    | <input type="checkbox"/> |
| Tomás Ospina       | Dev B · persistencia | Semilla de Mongo y adaptador de MongoDB (traducciones de la sección 1) | <input type="checkbox"/> |
| Santiago Rodríguez | Dev C · frontend     | SPA Vue y <code>src/api/</code> (adaptador HTTP y mock)                | <input type="checkbox"/> |

**Regla de cambio:** después del cierre, un cambio al contrato se propone en el repo (PR que edite este archivo), sube la versión (v1.1, v2.0...), se anota en el historial y lo aprueban los tres.